Cálculo RMS na ESP32

Root Mean Square • Fundamentos • Implementação • Otimização

Aula 16 • Sistemas Embarcados

Agenda

01 Fundamentos de RMS
Conceito, importância e aplicações

02 Definição Matemática
Fórmulas, interpretação física

03 Aquisição na ESP32
ADC, amostragem, quantização

04 Implementação
Algoritmo básico e código

05 Otimizações
Fixed-point, DMA, buffer circular

06 Aplicações Reais
Tensão AC, corrente, potência

O que é Valor RMS?

Root Mean Square

O valor RMS (**Raíz da Média dos Quadrados**) é uma medida estatística da magnitude de uma grandeza variável. Representa o valor equivalente DC que produz a mesma potência em uma carga resistiva.

Root **M**ean **S**quare

Raiz → Média → Quadrado

Potência

Equivalente DC que produz mesma potência

Sinais AC

Mede magnitude efetiva de sinais

Engenharia

Fundamental em sistemas elétricos

Valor Médio vs Pico vs RMS

Valor Médio

$|\bar{V}| = (1/T)\int|v(t)|dt$

Vpico × 0.637

Média aritmética dos valores absolutos

Uso: Retiificação, multímetros DC

Valor de Pico

Vpico = Vmáx

Vpico × 1.000

Amplitude máxima do sinal

Uso: Isolamento, proteção

Valor RMS

$Vrms = \sqrt{(1/T\int v^2dt)}$

Vpico × 0.707

Equivalência de potência DC

Uso: Potência, aquecimento

💡 **Para senoide pura: Vrms = Vpico / √2 ≈ 0.707 × Vpico**

Importância do RMS em Sinais Elétricos

Relação com potência: $P = V_{rms}^2 / R = I_{rms}^2 \times R$

Rede Elétrica

127V e 220V são valores RMS, não de pico. Vpico(127V) ≈ 180V

Aquecimento

Efeito térmico depende do valor RMS, não do valor médio ou pico

Medição

Multímetros true-RMS calculam RMS real, não aproximações

Proteção

Disjuntores e fusíveis respondem a corrente RMS equivalente

Definição Matemática do RMS

Sinal Contínuo

$$X_{rms} = \sqrt{\left[(1/T) \int x(t)^2 \, dt \right]}$$

T = período do sinal
Integração sobre um período completo

Sinal Discreto (ESP32)

$$X_{rms} = \sqrt{\left[(1/N) \sum x[n]^2 \right]}$$

N = número de amostras
Somatório de N amostras ao quadrado

1
Square
Eleve cada amostra ao quadrado

→

2
Mean
Calcule a média dos quadrados

→

3
Root
Extraía a raiz quadrada do resultado

Tipos de Sinais e Impacto no RMS

Sinal DC

Vrms = VDC

RMS = valor constante

Não há variação

Ex: Bateria 12V

Vrms = 12V

Senoidal

Vrms = Vp / √2

Fator de crista = √2

RMS ≈ 70.7% do pico

Ex: Rede 220Vac

Vpico = 311V

Arbitrário

Vrms = √(1/N ∑ x^2)

Sem fórmula simplificada

Cálculo numérico necessário

Ex: PWM, ruído

Depende da forma

Aquisição de Sinal na ESP32

ADC da ESP32

- Resolução: **12 bits** (0 - 4095)
- Faixa de entrada: **0 - 3.3V**
- Taxa máx: **~1 MSPS** (com DMA)
- Canais: **ADC1 (8 canais) + ADC2 (10 canais)**

Desafios

- **Ruído do ADC** - Não-linearidade intrínseca
- **Offset DC** - Varia entre canais e chips
- **Quantização** - 12 bits = 0.81mV/LSB
- **ADC2 + WiFi** - Conflito de uso simultâneo

Taxa de Amostragem → Teorema de Nyquist: fs ≥ 2 × fmax

Sinal 60Hz → fs ≥ 120Hz (recomendado: **≥10× = 600Hz**)

Prática: Coletar **N amostras por período completo** para RMS preciso (típico: 100-1000 amostras/ciclo)

Fluxo Completo do Cálculo RMS

1
Amostragem
ADC coleta
N amostras
$x[0]...x[N-1]$

→

2
Quadrado
Eleva cada
amostra
$x[n]^2$

→

3
Somatório
Acumula todos
os quadrados
$\sum x[n]^2$

→

4
Média
Divide pelo
número N
$\sum/N$

→

5
Raiz
Extrai raiz
quadrada
$\sqrt{(soma/N)}$

Resultado: $Xrms = \sqrt{[(x[0]^2 + x[1]^2 + ... + x[N-1]^2) / N]}$

Implementação & Código

Do algoritmo básico às otimizações de alto desempenho na ESP32

Implementação Básica — Algoritmo Direto

```
float calcRMS_basico(int pin, int N) {
  float soma = 0.0;

  for (int i = 0; i < N; i++) {
    float leitura = analogRead(pin);
    soma += leitura * leitura;  // x²
  }

  float media = soma / N;      // média
  return sqrt(media);        // raiz
}
```

Linha a Linha

1. Acumulador inicializado em 0
2. Loop N iterações
3. Lê amostra do ADC
4. Acumula o quadrado
5. Divide por N (média)
6. Retorna √(média)

Complexidade: O(N)

N leituras + N multiplicações + 1 divisão + 1 sqrt

Problemas da Abordagem Simples

Float é Lento

ESP32 sem FPU dedicada para double. Operações float = ~10x mais lentas que inteiros

~14 ciclos/mult

Bloqueante

analogRead() bloqueia a CPU durante a conversão ADC.
Não permite multitarefa

~10µs/leitura

Sem Tempo Real

Recalcula tudo a cada janela. Não atualiza continuamente. Latência proporcional a N

Latência: O(N)

Overflow

$4095^2 = 16.769.025$
Acumular N quadrados pode estourar 32 bits com N > 256

Risco: N > 256

Otimização 1: Aritmética Inteira (Fixed-Point)

Por que evitar float?

- ADC retorna **inteiros 12-bit** (0-4095)
- Multiplicação inteira: **1 ciclo** vs float: **~14 ciclos**
- Usar **uint64_t** para acumular, sqrt só no final

Ganho de Performance

float: **~850 µs** (1000 amostras)

int: **~120 µs** (1000 amostras)

Speedup: ~7x mais rápido

```
uint32_t calcRMS_inteiro(int pin, int N) {
  uint64_t somaQuad = 0;  // 64 bits evita overflow
  for (int i=0; i<N; i++) {
    uint32_t val = analogRead(pin);
    somaQuad += (uint64_t)val * val;
  }
  return (uint32_t)sqrt((double)somaQuad / N);
}
```

Otimização 2: Buffer Circular + RMS Incremental

Buffer Circular

```
[ x3 | x4 | x5 | x6 | x7 | x0 | x1 | x2 ]
                                ↑ idx
```

- Tamanho fixo N (potência de 2)
- Índice circular: **`idx = (idx+1) & (N-1)`**
- Sobrescreve amostra mais antiga

RMS Incremental (Janela Deslizante)

soma += novo² - antigo²

- **O(1) por amostra** (não O(N))
- Atualização contínua em tempo real
- Remove valor antigo, adiciona novo
- Ideal para monitoramento contínuo

Comparação: Batch vs Incremental

Batch (básico): **Recalcula N quadrados a cada atualização → O(N)**

Incremental: **1 subtração + 1 soma + 1 divisão + 1 sqrt → O(1)**

Código: RMS com Buffer Circular + Incremental

```c
#define BUF_SIZE 256              // potência de 2!
#define BUF_MASK (BUF_SIZE - 1)  // máscara para mod rápido

uint16_t buffer[BUF_SIZE];
uint64_t somaQuadrados = 0;
uint16_t idx = 0;

float rms_update(uint16_t novaAmostra) {
  uint16_t antiga = buffer[idx];    // recupera valor antigo
  buffer[idx] = novaAmostra;   // sobrescreve
  idx = (idx + 1) & BUF_MASK;         // mod rápido (bitwise)

  // Atualização incremental O(1)
  somaQuadrados += (uint64_t)novaAmostra*novaAmostra
                 - (uint64_t)antiga*antiga;

  return sqrtf((float)somaQuadrados / BUF_SIZE);
}
```

Otimização 3: DMA (Direct Memory Access)

ADC
Conversão automática

→

DMA
Transferência direta

→

Buffer
Memória RAM

→

CPU
Processa RMS

CPU Livre
ADC opera independente
CPU pode processar outras tarefas

Alta Taxa
Até 1 MSPS contínuo
Sem overhead de interrupção por amostra

Double Buffer
Ping-pong: DMA preenche um buffer enquanto CPU processa o outro

Estratégias Avançadas de Pré-Processamento

Remoção de Offset DC

ADC da ESP32 só lê 0-3.3V.

Sinais AC precisam de bias DC

(geralmente 1.65V = VCC/2)

Solução:

```
media = soma / N;
ac = amostra - media;
soma += ac * ac;
```

Filtro Digital

Média Móvel (FIR simples)

Suaviza ruído do ADC

Reduz flutuações espúrias

```
y[n] = (x[n] + x[n-1]
      + x[n-2] + x[n-3])
      >> 2; // div por 4
```

Normalização

Converter ADC raw para

volts reais:

V = (raw / 4095) × 3.3

Com sensor de corrente:

I = (V - Vref) / Sensib.

Calibração melhora

precisão em 2-5%

Código: RMS com Filtro + Remoção DC

```
float calcRMS_filtrado(int pin, int N) {
  // Passo 1: Coletar amostras e calcular média (offset DC)
  uint32_t somaDC = 0;
  uint16_t amostras[N];
  for(int i=0; i<N; i++) { amostras[i]=analogRead(pin); somaDC+=amostras[i]; }
  int16_t offset = somaDC / N;          // offset DC estimado

  // Passo 2: RMS da componente AC (sem DC)
  uint64_t somaQ = 0;
  for(int i=0; i<N; i++) {
    int32_t ac = (int32_t)amostras[i] - offset;
    somaQ += (int64_t)ac * ac;
  }
  return sqrtf((float)somaQ / N) * 3.3 / 4095.0; // em Volts
}
```

Precisão vs Desempenho: Trade-offs

Nº de Amostras

- N=64: rápido, baixa precisão
- N=256: bom equilíbrio
- N=1024: preciso, lento

→ Ideal: N = potência de 2

Freq. Amostragem

- Nyquist: fs ≥ 2×fmax
- RMS 60Hz: min 10 kSPS
- Ideal: 20-50 amostras/ciclo

→ ESP32 ADC: até ~100 kSPS

Tempo de Resposta

- Lento: batch (bloco completo)
- Rápido: janela deslizante
- Proteção: <20ms

→ Incremental = melhor

Comparação de Configurações (60Hz, ESP32 @ 240MHz)

```
N=64,   fs=10kHz  → T=6.4ms   Erro≈2.5%   Mem: 128B   Rápido
N=256,  fs=20kHz  → T=12.8ms  Erro≈0.8%   Mem: 512B   Equilibrado
N=1024,fs=50kHz  → T=20.5ms  Erro≈0.2%   Mem: 2KB    Preciso
```

Arquitetura: Fluxo Completo na ESP32

Abordagem Simples: ADC → CPU → RMS

Sensor Analógico → ADC analogRead() → CPU loop() → x^2 + soma acumular → sqrt() resultado

Abordagem Otimizada: ADC + DMA → Buffer → RMS

Sensor Analógico → ADC + DMA automático → Buffer A/B ping-pong → CPU: RMS incremental → Resultado filtrado

Simples: CPU 100% ocupada, bloqueante VS Otimizada: CPU livre, não-bloqueante, preciso

Casos Reais & Limitações

Aplicações práticas, desafios do hardware e soluções

Caso Real: Medição de Tensão AC

Circuito: Transformador + Divisor

- Trafo 220V → 9V (redução)
- Divisor resistivo: 0-3.3V
- Bias DC: 1.65V (VCC/2)
- Capacitor filtragem: 100nF

Conversão ADC → Volts Reais

1. Ler ADC (0-4095)
2. Subtrair offset DC (~2048)
3. Calcular RMS das amostras AC
4. Multiplicar por fator do trafo

Vac = Vrms_adc × (220/9) × 2

```
float medirTensaoAC() {
  const int N = 256;  const float FATOR_TRAFO = 48.89;  // 220/9*2 calibrado
  uint32_t somaDC=0;  uint64_t somaQ=0;  uint16_t buf[N];
  for(int i=0;i<N;i++){buf[i]=analogRead(34); somaDC+=buf[i]; delayMicroseconds(65);}
  int16_t dc = somaDC/N;
  for(int i=0;i<N;i++){int32_t ac=buf[i]-dc; somaQ+=(int64_t)ac*ac;}
  float Vrms = sqrtf((float)somaQ/N) * 3.3/4095.0 * FATOR_TRAFO;
  return Vrms;  // tensão AC real em Volts
}
```

Caso Real: Medição de Corrente

ACS712

Sensibilidade: **66-185 mV/A**
Range: **±30A**
Ref: **VCC/2**

SCT-013

Sensibilidade: **50mA/A**
Range: **0-100A**
Ref: **Burden R**

INA219

Sensibilidade: **Digital I2C**
Range: **0-3.2A**
Ref: **N/A**

Fórmula de Conversão (ACS712)

$$I_{rms} = Vrms_adc / Sensibilidade \quad | \quad Vrms_adc = RMS(V_{adc} - V_{ref})$$

⚠ **Calibração é essencial!** Compare com multímetro true-RMS e ajuste FATOR_CALIBRACAO

Caso Real: Monitoramento de Potência

Potência Aparente: $S = V_{rms} \times I_{rms}$ Potência Ativa: $P = (1/N) \sum v[n] \times i[n]$

Fator de Potência: $FP = P / S$ | Potência Reativa: $Q = \sqrt{(S^2 - P^2)}$

Sensor V (trafo) → **Sensor I (ACS712)** → **ESP32 ADC dual** → **Cálculo RMS + P** → **Wi-Fi MQTT**

Smart Meter
Medidor de energia residencial IoT

Industrial
Monitoramento de máquinas e motores

Solar
Inversor fotovoltaico eficiência e produção

Problemas e Limitações

⚠ Ruído do ADC

ADC da ESP32 tem ruído de ±20 LSB.

Afeta precisão em sinais pequenos.

✅ **Solução:**

Oversampling + média móvel
ADC externo (ADS1115) para alta precisão

⚠ Amostragem Insuficiente

Poucos pontos por ciclo causam

alias e erro no cálculo RMS.

✅ **Solução:**

fs ≥ 10× freq. do sinal
Para 60Hz: min 600 SPS (ideal 3kHz+)

⚠ Overflow em Cálculos

$4095^2 \times 1024 = 17.1$ bilhões

Estoura uint32_t (max 4.29 bi)!

✅ **Solução:**

Usar uint64_t para acumulador
Ou fixed-point com shifts

⚠ Precisão Numérica

float tem apenas 23 bits mantissa.

Erro acumula em somas longas.

✅ **Solução:**

Acumular em inteiro, converter
apenas no resultado final

Resumo & Conclusão

01

RMS = potência equivalente DC

Fundamental para medições elétricas reais

02

Discreto: $\sqrt{(1/N \times \sum x^2)}$

Versão implementada no microcontrolador

03

Remover offset DC é crítico

Bias do ADC polui o cálculo RMS

04

Otimizar: inteiro + incremental

Fixed-point e buffer circular para tempo real

05

Calibrar com referência

Multímetro true-RMS para ajuste fino

06

Trade-off: N vs velocidade

Mais amostras = preciso mas mais lento